

Regular tree languages over non-ranked alphabets^{*}

Anne Brüggemann-Klein
Institut für Informatik
Technische Universität München
Arcisstr. 21
80290 München, Germany

E-mail: brueggem@informatik.tu-muenchen.de
WWW: <http://www11.informatik.tu-muenchen.de>

Derick Wood
Department of Computer Science
Hong Kong University of Science & Technology
Clear Water Bay, Kowloon, Hong Kong
E-mail: dwood@cs.ust.hk
WWW: <http://www.cs.ust.hk/~dwood>

Version 0.3, April 19th 1998

1 Notation and definitions

Trees have at least one node; they have node labels taken from a finite alphabet Σ . We represent trees by ground terms, using the symbols in Σ as operators. Operators have no rank or arity, so they can have any number of operands, including none.

For example, the term $a(a(a())a())a(a())a())$ represents a complete binary tree of height 2, whose nodes all have the label a .

We denote alphabet symbols in Σ with a , strings over Σ (which we also call words) with w , and sets of words over Σ (which we also call string languages) with L . We denote trees

^{*}The work of the two authors was supported partially by a joint DAAD-HK grant. In addition, the work of the second author was supported under a grant from the Research Grants Council of Hong Kong.

with t , sets of trees (which we also call tree languages) with T , and sets of sets of trees (which we also call classes of tree languages) with $\mathcal{T}$. Subscripted and superscripted variables have the same types as their base names.

The greek letter ϵ denotes the empty string.

Definition 1.1 We define the set of nodes $nodes(t)$ of a tree t as a set of strings of natural numbers. The definition is by induction on t .

1. For a tree $a()$ with just one node labelled a we define

$$nodes(a()) = \{\epsilon\}.$$

2. For a tree $a(t_1, \dots, t_n)$, $n \geq 1$, we define

$$nodes(a(t_1, \dots, t_n)) = \bigcup_{1 \leq i \leq n} i \cdot nodes(t_i) \cup \{\epsilon\}.$$

The nodes of a tree viewed as a term correspond to occurrences of subterms. We denote nodes of trees with ν .

Definition 1.2 The root node $root(t)$ of a tree t is defined as ϵ . For each node ν of t we define the set of ν 's children $children(\nu)$ as the set of all nodes $\nu \cdot i$ in $nodes(t)$.

Note that the leaves and only the leaves of a tree, that is, the nodes corresponding to subterms of the form $a()$, have an empty set of children.

Definition 1.3 A node ν of a tree t is a leaf if and only if $children(\nu) = \emptyset$. The set of leaves of t is denoted by $leaves(t)$.

Definition 1.4 For each node ν of a tree t , the label of ν in Σ is denoted by $label(\nu)$. More precisely, we define for a tree $t = a(t_1, \dots, t_n)$, $n \geq 0$,

1. the label of the root node ϵ in t is a ,
2. the label of the node $i \cdot s$ in t is the label of the node s in t_i .

Definition 1.5 A two-way nondeterministic tree automaton (2NTA) is a triple (Q, δ, F) , so that Q is a finite set called the set of states, F is a subset of Q called the set of final or accepting states, and δ is a subset of $\Sigma \times Q^* \times Q \times \{u, d, s\}$ called the set of transitions, so that for each a in Σ , q in Q , and m in $\{u, d, s\}$, the set $\{w \in Q^* \mid \delta(a, w, q, m)\}$ is a regular set of words over the alphabet Q (regularity of the transition table).

We denote automata with M .

We define the computations of a 2NTA on a tree, that is, its behaviour, as a sequence of configurations. A configuration assigns a state of the automaton to each node in a so-called cut of the tree.

Definition 1.6 A cut of a tree t is a subset of $nodes(t)$, so that for each leaf node ν of t exactly one node on the path from the root to that node, that is, exactly one node that is represented by a prefix of ν , is in the cut.

We denote cuts with C .

Definition 1.7 A configuration of a 2NTA $M = (Q, \delta, F)$ operating on a tree t is a mapping $c : C \rightarrow Q$ from a cut C of t to the set of states Q of M .

We denote configurations with c .

Let ν be a node of a tree t and let $c : C \rightarrow Q$ be a configuration of the 2NTA M operating on t . If $children(\nu) \subseteq C$, then formally $c(children(\nu))$ is a subset of Q . We overload this notation so that $c(children(\nu))$ also denotes the sequence of states in Q which arises from the order of ν 's children in t .

Definition 1.8

1. A starting configuration of a 2NTA $M = (Q, \delta, F)$ operating on a tree t is a configuration $c : C \rightarrow Q$, so that $C = leaves(t)$ and $c(\nu)$ is any state q in Q so that $\delta(label(\nu), \epsilon, q, u)$.
2. A halting configuration is a configuration $c : C \rightarrow Q$ so that $C = \{root(t)\}$.
3. An accepting configuration is a halting configuration c so that $c(root(t)) \in F$.

Definition 1.9

1. A 2NTA $M = (Q, \delta, F)$ operating on a tree t makes a transition between two configurations $c_1 : C_1 \rightarrow Q$ and $c_2 : C_2 \rightarrow Q$ (in signs: $c_1 \rightarrow c_2$) if and only if it makes either an up transition or a down transition or a staying transition (to be defined below).
2. M makes an up transition from c_1 to c_2 if and only if t has a node ν so that
 - (a) $children(\nu) \subseteq C_1$,
 - (b) $C_2 = (C_1 \setminus children(\nu)) \cup \{\nu\}$,
 - (c) $\delta(label(\nu), c_1(children(\nu)), c_2(\nu), u)$, and
 - (d) c_1 is identical to c_2 on their domains' common subset $C_1 \cap C_2$.
3. M makes a down transition from c_1 to c_2 if and only if t has a node ν so that
 - (a) $\nu \in C_1$,
 - (b) $C_2 = (C_1 \setminus \{\nu\}) \cup children(\nu)$,
 - (c) $\delta(label(\nu), c_2(children(\nu)), c_1(\nu), d)$, and
 - (d) c_1 is identical to c_2 on their domains' common subset $C_1 \cap C_2$.
4. M makes a staying transition from c_1 to c_2 if and only if t has a node ν so that
 - (a) $\nu \in C_1$,

- (b) $C_2 = C_1$,
- (c) $\delta(\text{label}(\nu), c_1(\nu), c_2(\nu), s)$, and
- (d) c_1 is identical to c_2 on their domains' common subset $C_1 \setminus \{\nu\}$, which is equal to $C_2 \setminus \{\nu\}$.

Definition 1.10

1. A computation of a 2NTA M on a tree t from configuration c to configuration c' is a sequence of configurations $c_1, \dots, c_n$, $n \geq 1$, so that $c = c_1 \longrightarrow \dots \longrightarrow c_n = c'$.
2. An accepting computation of M on t is a computation from a starting configuration to an accepting configuration.

Definition 1.11

1. A tree t is recognizable by an automaton M if and only if there is an accepting computation of M on t .
2. The tree language $T(M)$ that is recognizable by a 2NTA M is the set of trees that are recognizable by M .

Definition 1.12

1. A nondeterministic ascending tree automaton (NATA) is a 2NTA $M = (Q, \delta, F)$ so that δ contains only transitions whose last component is u . For a NATA M we can consider δ a subset of $\Sigma \times Q^* \times Q$, dropping the last, constant component in the cross product.
2. A deterministic ascending tree automaton (DATA) is a NATA $M = (Q, \delta, F)$ so that for each a in Σ and w in Q^* there is at most one q in Q so that $\delta(a, w, q)$.
3. A tree automaton without further qualification is always a NATA.

Definition 1.13

1. A nondeterministic descending tree automaton (NDTA) is a 2NTA $M = (Q, \delta, I)$ so that δ contains only transitions whose last component is d . For a NDTA M we can consider δ a subset of $\Sigma \times Q^* \times Q$, dropping the last, constant component in the cross product.
2. A deterministic descending tree automaton (DDTA) is a NDTA $M = (Q, \delta, I)$ so that I is a singleton set and for each a in Σ and q in Q there is, for each natural number n , at most one w of length n in Q^* so that $\delta(a, w, q)$.

For a 2NTA $M = (Q, \delta, F)$, the set of transitions δ is a subset of $\Sigma \times Q^* \times Q \times \{u, d, s\}$. For a down transition (a, w, g, d) in δ , the interpretation is that M , when in state q and sitting on a node labelled a , can move down and label its children one after the other with the states in the string w , which is in Q^* . It might be confusing at a first glance that

in the transition table δ the next state for a down transition is given before the current state. We have chosen this notation for the sake of homogeneity. We are pointing this out here, since it might be particularly surprising with descending tree automata.

The behaviour of an ascending tree automaton, which is also a 2NTA, is precisely as when it is viewed as a 2NTA. For a descending tree automaton $M = (Q, \delta, I)$, the situation is slightly different. Computations between configurations are defined precisely the same as in the more general case of 2NTAs. However, the starting configuration for a tree t is any configuration $c : \{\text{root}(t)\} \rightarrow I$, a halting configuration is any configuration $c : \text{leaves}(t) \rightarrow Q$, and an accepting configuration is any halting configuration $c : \text{leaves}(t) \rightarrow Q$ so that for each leaf node ν of t the condition $\delta(\text{label}(\nu), \epsilon, c(\nu))$ holds. For obvious reasons, in the case of an descending tree automaton, the set I is called the set of initial states. For an ascending as well as for a descending tree automaton the language of recognizable trees are the trees for which the automaton can carry out a computation from a starting to an accepting state.

Definition 1.14 A tree language is called tree-regular if and only if it is recognizable by a tree automaton, that is, a NATA.

2 Languages recognizable by ascending and descending tree automata

The results in this section have analoga in the theory of tree automata over ranked alphabets. Each proof in this section is based on a proof for the ranked-alphabet case.

Theorem A *The tree languages that are recognizable by a descending nondeterministic tree automaton are precisely the tree-regular languages; that is, ascending and descending nondeterministic tree automata recognize precisely the same tree languages.*

PROOF Let $M = (Q, \delta, F)$ be an ascending tree automaton and let $M' = (Q, \delta, I)$ be the dual descending tree automaton, whose sets of states and transitions are the same as M 's and whose set of initial states is the same as M 's set of final states. We note that for any tree t the starting configurations of M on t are precisely the accepting configurations of M' on t and that the accepting configurations of M on t are precisely the starting configurations of M' on t . Furthermore, for any two configurations c_1 and c_2 on t , M makes a transition from c_1 to c_2 when operating on t if and only if M' makes the reverse transition from c_2 to c_1 . Hence, accepting computations of M on t are precisely the reverse of accepting computations of M' on t . Therefore, M and M' recognize the same tree language. $\square$

Theorem B *Each tree-regular language is recognizable by an ascending deterministic tree automaton; that is, ascending deterministic and nondeterministic tree automata recognize precisely the same tree languages.*

PROOF We apply the well-known subset construction for string automata or tree automata over ranked alphabets to tree automata over non-ranked alphabets.

Let $M = (Q, \delta, F)$ be an ascending nondeterministic tree automaton. We define an ascending deterministic tree automaton $M' = (Q', \delta', F')$ that recognizes the same tree languages as M .

Let Q' be the set of Q 's subsets and let F' be the set $\{S \subseteq Q \mid S \cap F \neq \emptyset\}$.

For each symbol a in Σ , each word $W = X_1 \dots X_n$ over Q' , $n \geq 0$, and each state S in Q' , we define $\delta'(a, W, S)$ if and only if

$$S = \{q \mid \text{there are } q_i \text{ in } X_i, 1 \leq i \leq n, \text{ so that } \delta(a, q_1 \dots q_n, q)\}.$$

In particular, $\delta'(a, \epsilon, S)$ if and only if

$$S = \{q \mid \delta(a, \epsilon, q)\}.$$

We have to demonstrate the regularity of the transition table; that is, for each a in Σ and each $S \in Q'$, the set

$$L'_{a,S} = \{X_1 \dots X_n \text{ in } Q'^* \mid \delta'(a, X_1 \dots X_n, S)\}$$

is regular.

First, we note that $X_1 \dots X_n \in L'_{a,S}$ if and only if the following two conditions hold:

1. For each q in S , there are q_i in X_i , $1 \leq i \leq n$, so that $\delta(a, q_1 \dots q_n, q)$.
2. For each q_i in X_i , $1 \leq i \leq n$, there is a q in S so that $\delta(a, q_1 \dots q_n, q)$.

Since for each a in Σ and each q in Q the set

$$\{q_1 \dots q_n \text{ in } Q^* \mid \delta(a, q_1 \dots q_n, q)\}$$

is a regular string language, readers might be tempted to conclude at this point that $L'_{a,S}$ is also regular since it appears to be built from unions and intersections of regular languages. However, those unions and intersections are unbounded. Therefore, we need to present a more sophisticated argument that involves string homomorphisms and, in particular, the well-known fact that homomorphic images of regular sets are also regular. Additionally, we consider the $\cap$ -homomorphic image

$$h^\cap(L) = \{w \mid \text{each } v \text{ for which } h(v) = w \text{ is in } L\}$$

of a language L with respect to the homomorphism h and note that the $\cap$ -homomorphic images of regular sets are also regular.

Now we observe that

$$L''_{a,q} = \{(q_1, X_1) \cdots (q_n, X_n) \mid \delta(a, q_1 \cdots q_n, q)\}$$

is a regular string language over the alphabet

$$Q'' = \{(q, X) \mid q \in X \in Q'\}.$$

Furthermore, we consider the homomorphism $h : Q''^* \longrightarrow Q'^*$ that maps each (q, X) in Q'' onto its second component X .

Since $L'_{a,S}$ is the intersection of the two sets

$$\bigcap_{q \in S} h(L''_{a,q})$$

and

$$h^\cap(\bigcup_{q \in S} L''_{a,q}),$$

we may now correctly conclude that $L'_{a,S}$ is string-regular.

Having established the regularity of the transition table of $M' = (Q', \delta', F')$, it is now obvious that M' is indeed an ADTA and that M and M' recognize the same tree languages. $\square$

Theorem C *Some tree languages are tree-regular but not recognizable by any descending deterministic tree automaton.*

PROOF Any DDTA that recognizes the two trees $a(a(), a())$ and $a(b(), b())$ also recognizes the trees $a(a(), b())$ and $a(b(), a())$. On the other hand, the two-element tree language $\{a(a(), a()), a(b(), b())\}$ is recognizable by some DNTA. $\square$

3 Local tree languages and homomorphic images

Definition 3.1 A tree grammar G over an alphabet Σ is a tuple (P, I) so that P is a subset of $\Sigma \times \Sigma^*$, for each a in Σ the set $\{w \mid a \longrightarrow w \text{ in } P\}$ is string-regular, and I is a non-empty subset of Σ . We call P the set of productions and I the set of start symbols.

A tree grammar is very similar to an extended context-free grammar; there are just two differences: First, a tree grammar can have more than one start symbol, and second, with tree grammars, there is no distinction between terminal and non-terminal symbols.

We denote tree grammars with G .

Definition 3.2 The derivation trees of a tree grammar $G = (P, I)$ with root label a , $a \in \Sigma$, are defined inductively: For each production $a \longrightarrow w$, $w = a_1 \cdots a_n$, $n \geq 0$, and any derivation trees t_1 with root label a_1 , $\dots$, t_n with root label a_n , the tree $a(t_1, \dots, t_n)$ is a derivation tree of G with root label a . A derivation tree of G is a derivation tree with a root label in I .

Note that a -labelled leaves in a derivation tree correspond to productions $a \longrightarrow \epsilon$ in the grammar.

Definition 3.3 A tree language is called tree-local if and only if it is the set of derivation trees for a tree grammar.

Definition 3.4 A tree homomorphism is based on a mapping from one alphabet to another and is extended to trees in the canonical way.

Theorem D *A tree language is tree-regular if and only if it is the homomorphic image of a tree-local language.*

PROOF The following observation aids in clarifying the connection between tree automata and tree grammars:

One-way tree automata visit each node of a tree exactly once during an accepting computation, so the whole computation can be represented by adding an additional Q -label to each Σ -labelled node.

More precisely, let $M = (Q, \delta, I)$ be a descending nondeterministic automaton and let t be a tree. We add a second, Q -component to the labels of t , thus constructing a number of $\Sigma \times Q$ -labelled trees, as follows:

1. Each state q in I can be added as a second component to the root of t .
2. If q has been added to an a -labelled node ν of t , if ν has precisely n children, and if $\delta(a, a_1 \cdots a_n, q)$, then a_i can be added to the i th child of ν , $1 \leq i \leq n$.

We only consider those $\Sigma \times Q$ -labelled trees t' thus constructed from t whose every node has been additionally labelled and at whose every (a, q) -labelled leaf the condition $\delta(a, \epsilon, q)$ holds.

There is a one-to-one correspondence between accepting computations of M on t and the thus constructed $\Sigma \times Q$ -labellings of t , which we call M 's accepting computation trees corresponding to t . In particular, if t is not recognizable by M , then no corresponding accepting computation $\Sigma \times Q$ -tree can be constructed from t and vice versa.

After these preliminaries, we now proceed to the proof proper.

For an descending nondeterministic tree automaton $M = (Q, \delta, I)$ on the one hand let us now define the grammar $G = (P, \Sigma \times I)$ over the alphabet $\Sigma' = \Sigma \times Q$ as follows:

$$P = \{(a, q) \longrightarrow (a_1, q_1) \cdots (a_n, q_n) \mid n \geq 0, a_1, \dots, a_n \in \Sigma, \delta(a, q_1 \cdots q_n, q)\}.$$

The parse trees of G correspond precisely to the accepting computation trees of M . Therefore, we define h on Σ' as the projection onto the first, label component, and get the intended result, namely that the h -images of G 's parse trees are precisely the trees recognizable by M .

For a tree grammar $G = (P, I)$ on the other hand, let us now define the descending nondeterministic tree automaton $M = (\Sigma', \delta, I)$ as follows: For each a in Σ , $q_1 \cdots q_n$ in Σ'^* , q in Σ let $\delta(a, q_1 \cdots q_n, q)$ if and only if

1. $h(q) = a$,
2. the production $q \longrightarrow q_1 \cdots q_n$ is in P .

The accepting computation trees of M are labelled with pairs (a, q) in $\Sigma \times \Sigma'$ so that $h(q) = a$. If we drop the first, Σ -labelled components from the accepting computation trees of M , the results are precisely the parse trees of G . Hence, the trees recognizable by M are precisely the h -images of G 's parse trees. $\square$

Theorem E *For each tree-regular tree language there is a uniquely defined minimal tree-local superset.*

Theorem F *Tree-local tree languages do not form a Boolean algebra.*

4 Characterizable tree languages

Definition 4.1 An algebra $\mathcal{A} = (A, S)$ consists of a set A and a set of functions S . Each function s in S has an arity or rank $r(s)$ in $\mathbf{N}_0$ so that $s : A^{r(s)} \longrightarrow A$.

Definition 4.2 Let $\mathcal{A} = (A, S)$ be an algebra. An equivalence relation $\sim$ on A is called an S -congruence if and only if any s in S and $a_1, \dots, a_{r(s)}, b_1, \dots, b_{r(s)}$ in A satisfy the following condition: If $a_i \sim b_i$, $1 \leq i \leq r(s)$, then $s(a_1, \dots, a_{r(s)}) \sim s(b_1, \dots, b_{r(s)})$; that is, an $\mathcal{A}$ -congruence is an equivalence relation on A that is compatible with the functions in S .

Definition 4.3 An equivalence relation is of finite index if it has a finite number of equivalence classes.

Definition 4.4 Let $\mathcal{A} = (A, S)$ be an algebra. A subset L of A is S -characterizable if and only if there is a finite S -congruence on A so that L is the union of some of its equivalence classes.

Proposition 4.1 *Let $\mathcal{A} = (A, S)$ be an algebra. For any finite number of S -characterizable subsets of A there is a single S -congruence that characterizes all of them simultaneously.*

Proposition 4.2 *Let $\mathcal{A} = (A, S)$ be an algebra. The subsets of A that are S -characterizable form a Boolean algebra.*

Definition 4.5 For a in Σ let the string functions p_a and s_a be defined as follows: $p_a : \Sigma^* \rightarrow \Sigma^*$, $p_a(w) = aw$, $s_a : \Sigma^* \rightarrow \Sigma^*$, $s_a(w) = wa$. Furthermore, let $\cdot$ denote string concatenation.

The following well-known characterization for string languages is commonly attributed to Myhill and Nerode:

Proposition 4.3 *Let S be any of the following three sets of functions: $\{\cdot\}$, $\{p_a \mid a \in \Sigma\}$, $\{s_a \mid a \in \Sigma\}$. Then, a subset L of Σ^* is S -characterizable if and only if L is string-regular.*

A $\{p_a \mid a \in \Sigma\}$ -congruence is called a left congruence and a $\{s_a \mid a \in \Sigma\}$ -congruence is called a right congruence on Σ^* .

Definition 4.6 For trees t and t' , $t' = a(t_1, \dots, t_n)$, we define $t \downarrow t'$ as the tree $a(t, t_1, \dots, t_n)$, inserting t as the first child of t' .

A theorem on trees in the spirit of Myhill and Nerode.

Theorem G *A set of trees T is tree-regular if and only if T is $\downarrow$ -characterizable.*

PROOF In the first part of the proof let T be a tree-regular set of trees and let $M = (Q, \delta, F)$ be a tree automaton that recognizes T . We construct an equivalence relation $\approx$ that $\downarrow$ -characterizes T .

The regularity condition for the transition table of M ensures that for each a in Σ and q in Q the set

$$\{w \text{ in } Q^* \mid \delta(a, w, q)\}$$

is string-regular. Since there are only finitely many such sets, we can find a single $\{p_a \mid a \in \Sigma\}$ -congruence $\sim$ on Σ that characterizes them all simultaneously.

A necklace of a tree t is an equivalence class $[w]_\sim$, $w \in Q^*$, so that M , when operating on t , can generate w as the sequence of states for the children of t 's root.

We define trees t_1 and t_2 to be equivalent ($t_1 \approx t_2$) if and only if t_1 and t_2 have identical root labels and identical sets of necklaces.

First of all, $\approx$ is an equivalence relation on the set of trees, and it is of finite index.

Furthermore, if $t_1 \approx t_2$ and $t_1 \in T$, then $t_2 \in T$. The reason is that the sets of states that M , when operating on t_1 resp. t_2 , yields for the roots of t_1 resp. t_2 only depend on the root labels and on the sets of necklaces.

It remains to be demonstrated that $\approx$ is a $\downarrow$ -congruence. For that let $t_1 \approx t_2$, $t_3 \approx t_4$. We can see immediately that $t_1 \downarrow t_3$ and $t_2 \downarrow t_4$ have identical root labels. Furthermore, the necklaces of $t_1 \downarrow t_3$ are precisely the necklaces $[p_q(w)]_\sim$ so that q is a state that M when operating on t_1 yields for t_1 's root and $[w]_\sim$ is a necklace of t_3 . The analogous argument holds for t_2 and t_4 , so the sets of necklaces for $t_1 \downarrow t_3$ and $t_2 \downarrow t_4$ are the same.

We have thus demonstrated that $\approx \downarrow$ -characterizes T .

In the second part of the proof let now $\approx$ be an equivalence relation that $\downarrow$ -characterizes the set of trees T . We construct a tree automaton $M = (Q, \delta, F)$ that recognizes T .

First, let

$$Q = \{[t]_\approx \mid t \text{ is a tree}\}.$$

Since $\approx$ is of finite index, the set of states Q is finite.

Second, let

$$F = \{[t]_\approx \mid t \in T\}.$$

Since $T = \bigcup \{[t]_\approx \mid t \in T\}$, if $t_1 \approx t_2$ and $t_1 \in T$ then $t_2 \in T$.

Finally, let

$$\delta = \{(a, [t_1]_\approx \cdots [t_n]_\approx, [a(t_1, \dots, t_n)]_\approx)\}.$$

Since $a(t_1, \dots, t_n) = t_1 \downarrow (\dots (t_{n-1} \downarrow (t_n \downarrow a)) \dots)$ and since $\approx$ is a $\downarrow$ -congruence, the transition table δ is well-defined.

The tree automaton $M = (Q, \delta, F)$ is deterministic. When operating on the tree t , the automaton M yields $[t]_\approx$ as the state for t 's root. Hence, M recognizes t .

We still have to demonstrate the regularity condition for M 's transition table δ :

We choose, for each state q in Q , a representative tree t_q in q , so that $q = [t_q]_\approx$.

For each a in Σ and each tree t we demonstrate that the set

$$\{q_1 \cdots q_n \mid a(t_{q_1}, \dots, t_{q_n}) \approx t\}$$

is string-regular over Q , using the Myhill-Nerode theorem for string languages. We define the equivalence relation $\sim$ over Q -strings as follows:

$$q_1 \cdots q_m \sim q'_1 \cdots q'_n$$

if and only if

$$a(t_{q_1}, \dots, t_{q_m}) \approx a(t_{q'_1}, \dots, t_{q'_n}).$$

Since $\approx$ is of finite index, so is $\sim$.

Next we demonstrate that $\sim$ is a p_q -congruence for each q in Q . So let $q_1 \cdots q_m \sim q'_1 \cdots q'_n$. Then

$$a(t_{q_1}, \dots, t_{q_m}) \approx a(t_{q'_1}, \dots, t_{q'_n})$$

and, since $\approx$ is a $\downarrow$ -congruence,

$$\begin{aligned} a(t_q, t_{q_1}, \dots, t_{q_m}) &= t_q \downarrow a(t_{q_1}, \dots, t_{q_m}) \\ &\approx t_q \downarrow a(t_{q'_1}, \dots, t_{q'_n}) \\ &= a(t_q, t_{q'_1}, \dots, t_{q'_n}). \end{aligned}$$

Hence,

$$p_q(q_1 \cdots q_m) = qq_1 \cdots q_m \sim qq'_1 \cdots q'_n = p_q(q'_1 \cdots q'_n).$$

Finally, we demonstrate that $\sim$ characterizes the set

$$\{q_1 \cdots q_n \mid a(t_{q_1}, \dots, t_{q_n})\}.$$

This, however, is obvious, since for each $q_1 \cdots q_m$ that belongs to this set and for each $q'_1 \cdots q'_n$ that is $\sim$ -equivalent to $q_1 \cdots q_m$ the string $q'_1 \cdots q'_n$ belongs to this set as well. Hence,

$$\{q_1 \cdots q_n \mid a(t_{q_1}, \dots, t_{q_n})\}$$

is the union of $\sim$ -equivalence classes. □

Theorem H *The tree-regular tree languages form a Boolean algebra.*

5 Top congruences

Definition 5.1 A pointed tree (sometimes also called a tree with a handle or a handled tree) is a tree over an extended alphabet $\Sigma \cup \{X\}$ so that precisely one node is labelled with the variable X and that node is a leaf.

Definition 5.2 If t is a pointed tree and t' is a (pointed or non-pointed) tree, we can concatenate t and t' by replacing the node labelled X in t with the root of t' . The result is the (pointed or non-pointed) tree tt' .

Definition 5.3 Let T be a tree language. Trees t_1 and t_2 are top congruent with respect to T ($t_1 \sim_T t_2$ or simply $t_1 \sim t_2$) if and only if for each pointed tree t the following condition holds:

$$tt_1 \in T \text{ if and only if } tt_2 \in T.$$

The tree functions p_t that map each tree t' to $p_t(t') = tt'$ are, for pointed trees t , the tree analog of the string functions p_a , $a \in \Sigma$. The top congruence on trees is the tree analog of the left congruence on strings.

Lemma 5.1 *The top congruence is an equivalence relation on trees; it is a congruence with respect to concatenations of trees to pointed trees.*

PROOF To verify the congruence condition, let t be a pointed tree. We have to demonstrate that $t_1 \sim_T t_2$ implies that $tt_1 \sim_T tt_2$. Since tree concatenation is associative, for any trees t_1 and t_2 so that $t_1 \sim_T t_2$ and for any pointed tree t'

$$t'(tt_1) = (t't)t_1 \sim_T (t't)t_2 = t'(tt_2).$$

Hence, $tt_1 \sim_T tt_2$. □

Definition 5.4 The top index of a tree language T is the number of $\sim_T$ -equivalence classes.

Lemma 5.2 *Each tree-regular tree language is of finite top index.*

PROOF Let T be a tree-regular language that is recognizable by the tree automaton M . For any trees t_1 and t_2 , if the automaton M when operating on t_1 yields the same states in a halting configuration as when operating on t_2 , then $t_1 \sim_T t_2$. Since M 's set of states is finite, the equivalence relation $\sim_T$ is of finite index. □

In contrast to string languages, that a tree language is of finite index is not sufficient for tree-regularity. As we will see, a second condition, namely regularity of so-called local views, has to be satisfied as well.

Definition 5.5 Let T be a tree language, let a be a symbol in Σ , let t be a pointed tree, let T_f be a finite set of trees, and let Σ_{T_f} be a finite alphabet, so that each symbol x in Σ_{T_f} corresponds to precisely one tree t_x in T_f . The local view of T with respect to t , a , and T_f is the string language

$$V_{t,a,T_f}(T) = \{x_1 \cdots x_n \in \Sigma_{T_f}^* \mid ta(t_{x_1}, \dots, t_{x_n}) \in T\}.$$

Lemma 5.3 *For each tree-regular tree language, all its local views are regular string languages.*

PROOF Let T be a tree-regular language recognizable by the tree automaton $M = (Q, \delta, F)$, let a be a symbol in Σ , let t be a pointed tree, and T_f be a finite set of trees, Σ_{T_f} be a finite alphabet, so that each symbol x in Σ_{T_f} corresponds to precisely one tree t_x in T_f .

For each q in Q let $M_q = (Q, \delta_q, F)$ so that $\delta_q = \delta \cup \{(X, \epsilon, q)\}$; that is, M_q is identical to M with the exception of one additional element in M_q 's transition table. The automaton M_q operates on $\Sigma \cup \{X\}$ -labelled trees.

A computation of M on a tree tt' can be divided into two phases, namely a computation of M on t' , resulting in some state q at the root of t' , and a computation of M_q on the pointed tree t . The automaton M recognizes a tree tt' if and only if M , when operating on t' , makes a sequence of transitions from any starting configuration to a halting configuration whose root label is q and if M_q recognizes t .

For x in Σ_{T_f} let Q_x be the set of states q in Q so that M , when operating on t_x , makes a sequence of transitions from any starting configuration to a halting configuration whose root label is q . Because of the regularity condition of M 's transition table, the set

$$\{(x_1, q_1) \cdots (x_n, q_n) \mid q_i \in Q_{x_i}, \delta(a, q_1 \cdots q_n, q), M_q \text{ recognizes } t\}$$

is the inverse homomorphic image of a string-regular set and, hence, is itself string-regular. But then the local view V_{t,a,T_f} is the homomorphic image of a string-regular set and, hence, is also string-regular. $\square$

Example Let

$$T = \{c(t_1, \cdots t_n) \mid \text{label}(\text{root}(t_1)) \cdots \text{label}(\text{root}(t_n)) \in \{a^l b^l \mid l \geq 1\}\}.$$

The tree language T is of index 2; its two equivalence classes are the sets of trees whose root labels are a respectively b . The local view of T with respect to the pointed tree $X()$, symbol c , and the finite set of trees $\{a(), b()\}$ is the non-regular set of strings $\{a^l b^l \mid l \geq 1\}$. Hence, T is of finite top index but not tree-regular.

Theorem I *A tree language is tree-regular if and only if it is of finite top index and all its local views are regular string languages.*

PROOF The two previous lemmata establish already one of the two if clauses. So let T be a tree languages of finite top index so that all its local views are regular string languages. Let $\sim$ denote the top congruence with respect to T and let $[t]$ denote the equivalence class of a tree t with respect to the top congruence. We have to construct a tree automaton $M = (Q, \delta, F)$ that recognizes T .

The first part of the proof is a straight-forward adaption of its string-languages counterpart. The second part deals with the regularity of the transition table.

Let

$$Q = \{[t] \mid t \text{ is a tree}\}$$

and let

$$F = \{[t] \mid t \in T\}.$$

We note that $t_1 \sim t_2$ implies that $t_1 \in T$ if and only if $t_2 \in T$, so $[t]$ either only consists of trees in T or of trees not in T .

Finally, let δ consist of the triples

$$(a, [t_1] \cdots [t_n], [a(t_1, \dots, t_n)]).$$

We note that δ is well-defined; that is, if $t_1 \sim t'_1, \dots, t_n \sim t'_n$, then $a(t_1, \dots, t_n) \sim a(t'_1, \dots, t'_n)$. The argument is incremental: Let t be a pointed tree. Then

$$\begin{aligned} ta(t_1, \dots, t_n) \in T &\Leftrightarrow ta(X, t_2, \dots, t_n)t_1 \in T \\ &\Leftrightarrow ta(X, t_2, \dots, t_n)t'_1 \in T \\ &\Leftrightarrow ta(t'_1, t_2, \dots, t_n) \in T \\ &\Leftrightarrow ta(t'_1, X, t_3, \dots, t_n)t_2 \in T \\ &\Leftrightarrow ta(t'_1, X, t_3, \dots, t_n)t'_2 \in T \\ &\Leftrightarrow ta(t'_1, t'_2, t_3, \dots, t_n) \in T \\ &\Leftrightarrow \vdots \\ &\Leftrightarrow ta(t'_1, \dots, t'_n) \in T. \end{aligned}$$

Hence, $a(t_1, \dots, t_n) \sim a(t'_1, \dots, t'_n)$.

Our last task is to demonstrate the regularity condition for M 's transition table δ . So for each q in Q let t_q in q be a representative of q ; that is, $[t_q] = q$. Furthermore, let $T_f = \{t_q \mid q \in Q\}$. We may then choose $\Sigma_{T_f} = Q$. Finally let $\hat{T}$ be a finite set of pointed trees that discriminates between the finitely many $\sim$ -equivalence classes. More precisely, $t_1 \sim t_2$ if and only if, for each t in $\hat{T}$, the tree tt_1 is in T if and only if tt_2 is in T .

Then $\delta(a, q_1 \cdots q_n, q)$ if and only if $a(t_{q_1}, \dots, t_{q_n}) \sim t_q$; that is, if and only if for each t in $\hat{T}$ the tree $ta(t_{q_1}, \dots, t_{q_n})$ is in T if and only if the tree tt_q is in T . Hence, $\delta(a, q_1 \cdots q_n, q)$ if and only if $q_1 \cdots q_n$ is in the intersection of the sets $V_{t, a, T_f}(T)$ so that $t \in \hat{T}$ and $tt_q \in T$ and of the complements of the sets $V_{t, a, T_f}(T)$ so that $t \in \hat{T}$ and $tt_q \notin T$. This intersection, however, is a string-regular set, which establishes the regularity condition of M 's transition table. $\square$

6 Languages recognizable by two-way tree automata

Theorem J *Each tree language that is accepted by a 2NTA is tree-regular.*

PROOF Let $M = (Q, \delta, F)$ be a 2NTA that recognizes the tree language T . We demonstrate that T is of finite top index and that all its local views are string-regular.

For each tree t let Q_t be the union of the two sets

$$\{q \in Q \mid \begin{array}{l} c_1 \longrightarrow c_2, \text{ } c_1 \text{ a starting configuration for } M \text{ on } t, \\ c_2 \text{ a halting configuration for } M \text{ on } t, \text{ } c_2(\text{root}(t)) = q \end{array}\}$$

and

$$\{(q_1, q_2) \in Q \times Q \mid \begin{array}{l} c_1 \longrightarrow c_2, \text{ } c_1 \text{ and } c_2 \text{ halting configurations for } M \text{ on } t, \\ c_1(\text{root}(t)) = q_1, \text{ } c_2(\text{root}(t)) = q_2 \end{array}\}.$$

The set $\{Q_t \mid t \text{ is a tree}\}$ is finite. Furthermore, if $Q_{t_1} = Q_{t_2}$, then $t_1 \sim_T t_2$. Hence, T is of finite top index.

Now let t be a pointed tree, T_f be a finite set of trees, Σ_{T_f} be a finite alphabet so that each symbol x in Σ_{T_f} corresponds to precisely one tree t_x in T_f .

We demonstrate that the local view of T with respect to t , a , and T_f , namely

$$V_{t,a,T_f}(T) = \{x_1 \cdots x_n \mid ta(t_{x_1}, \dots, t_{x_n}) \in T\}$$

is string-regular.

It is sufficient to demonstrate that for each p and q in Q the sets

$$\begin{aligned} X_p = \{ & x_1 \cdots x_n \in \Sigma_{T_f}^* \mid \\ & c_1 \longrightarrow c_2, \\ & c_1 \text{ is starting configuration of } M \text{ on } a(t_{x_1}, \dots, t_{x_n}), \\ & c_2 \text{ is halting configuration of } M \text{ on } a(t_{x_1}, \dots, t_{x_n}), \\ & c_2(\text{root}(a(t_{x_1}, \dots, t_{x_n}))) = p, \\ & \text{there is no other halting configuration in the computation } c_1 \longrightarrow c_2 \} \end{aligned}$$

and

$$\begin{aligned} X_{p,q} = \{ & x_1 \cdots x_n \in \Sigma_{T_f}^* \mid \\ & c_1 \longrightarrow c_2, \\ & c_1 \text{ and } c_2 \text{ are halting configuration of } M \text{ on } a(t_{x_1}, \dots, t_{x_n}), \\ & c_1(\text{root}(a(t_{x_1}, \dots, t_{x_n}))) = p, \\ & c_2(\text{root}(a(t_{x_1}, \dots, t_{x_n}))) = q, \end{aligned}$$

there is no other halting configuration in the computation $c_1 \longrightarrow c_2$

are string regular, since V_{t,a,T_f} is a finite union of finite intersections of those sets.

First the string $x_1 \cdots x_n$ is in X_p if and only if there are $p_1, \dots, p_n$ in Q so that M , when operating on t_{x_i} , starting in a starting configuration, eventually reaches a halting configuration c so that $c(\text{root}(t_{x_i})) = p_i$ and $\delta(a, p_1 \cdots p_n, p, u)$. Hence, X_p is the homomorphic image of a string-regular set and therefore itself string-regular.

Second, let $X_{p,q}^s$ be the subset of $X_{p,q}$ in which the computation $c_1 \longrightarrow c_2$ (compare definition of $X_{p,q}$) takes just one computation step, and let $X_{p,q}^m$ be the subset of $X_{p,q}$ in which the computation $c_1 \longrightarrow c_2$ takes more than one computation steps. Then, $X_{p,q}$ is the (not necessarily disjoint) union of $X_{p,q}^s$ and $X_{p,q}^m$. We demonstrate that both subsets are string-regular.

We note

$$X_{p,q}^s = \{x_1 \cdots x_n \in \Sigma_{T_f}^* \mid \delta(a, p, q, s)\}.$$

Hence, $X_{p,q}^s$ only depends on a and is either empty or equal to $\Sigma_{T_f}^*$. In particular, $X_{p,q}^s$ is regular.

Furthermore, $x_1 \cdots x_n \in X_{p,q}^m$ if and only if there are $p_1, \dots, p_n, q_1, \dots, q_n$ in Q so that $\delta(a, p_1 \cdots p_n, p, d)$ and M , when operating on t_{x_i} makes a computation from a halting configuration with root label p_i to a halting configuration with root label q_i , and $\delta(a, q_1 \cdots q_n, q, u)$. Hence, $X_{p,q}^m$ is the homomorphic image of a string-regular language and, therefore, itself regular. $\square$

7 Tree-regular expressions

8 Tree automata with endmarkers

We introduce two new symbols, $\top$ and $\perp$, which serve as endmarkers on tree. So let Σ^e be the disjoint union of Σ and $\{\top, \perp\}$.

Definition 8.1 A Σ^e -labelled tree is called endmarked if and only if

1. its root is labelled $\top$,
2. its leaves are labelled $\perp$,
3. its root is unary,
4. its leaves have no siblings,
5. each node that is not the root nor a leaf is labelled with a symbol in Σ , and

6. there is at least one Σ -labelled node.

Obviously, there is a one-to-one correspondence between Σ -labelled trees and endmarked trees. If we add to the Σ -labelled tree t a new root labelled $\top$ and to each leaf a new child labelled $\perp$, the result is the endmarked version t^e . For each tree language T we define the endmarked version $T^e = \{t^e \mid t \in T\}$.

Theorem K *If the tree language T over the extended alphabet Σ^e is tree-regular, then so is its endmarked subset.*

PROOF Let the tree automaton $M = (Q, \delta, F)$ recognize the tree language T over the extended alphabet Σ^e . We define a tree automaton $M' = (Q', \delta', F')$ that recognizes precisely the endmarked trees in T .

We construct δ' from δ :

First, we remove all transitions on the symbols $\top$ and $\perp$ and all transitions on leaves from δ ; that is all transitions of the form (a, ϵ, q) whose second component is the empty string. The only transitions we'll add later on are transitions on $\perp$ -labelled leaves or on $\top$ -labelled nodes, the latter being transitions into a state that disables further up moves. Therefore, any tree recognizable by M' has $\perp$ -labelled leaves and a $\top$ -labelled root; all other nodes are Σ -labelled.

Next, we add back all transitions on $\perp$ -labelled leaves, yet we mark the target states, so that further transitions can ensure that leaves have no siblings and that parents of leaves are Σ -labelled.

Finally, for each transition so that $\delta(\top, q, q')$ and $q' \in F$ we add a transition $(\top, q, q_f)$ so that q_f is a new and final state in Q' , thus ensuring that the roots of M' -recognizable trees are unary and $\top$ -labelled.

More precisely, let Q' be the disjoint union of Q , singleton $\{q_f\}$, and $Q \times \{q_s\}$, and let $F' = \{q_f\}$. Furthermore, let δ' be

$$\begin{aligned} \delta \cap (\Sigma \times Q^* \times Q) &\setminus \{(a, \epsilon, q) \mid a \in \Sigma^*, q \in Q\} \\ &\cup \{(\perp, \epsilon, (q, q_s)) \mid \delta(\perp, \epsilon, q)\} \\ &\cup \{(a, (q, q_s), q') \mid \delta(a, q, q')\} \\ &\cup \{(\top, q, q_f) \mid \delta(\top, q, q') \text{ and } q' \in F\}. \end{aligned}$$

□

Theorem L *Let T be a tree language over the alphabet Σ . Then T is tree-regular if and only if T^e is tree-regular.*

PROOF First, let T be tree-regular and let the tree automaton $M = (Q, \delta, F)$ recognize T . We define a tree automaton $M' = (Q', \delta', F')$ so that the endmarked trees that

M' recognizes are precisely the trees in T^e .

Let Q' be the disjoint union of Q and the singleton $\{q_s\}$ and $F' = F$. Furthermore, let

$$\begin{aligned} \delta' = \delta \quad &\cup \quad \{(\perp, \epsilon, q_s)\} \cup \{(a, q_s, q) \mid \delta(a, \epsilon, q)\} \\ &\cup \quad \{(\top, q, q) \mid q \in Q\}. \end{aligned}$$

Second, let T^e be tree-regular and let the tree automaton $M = (Q, \delta, F)$ recognize T^e . We define a tree automaton $M' = (Q', \delta', F')$ so that M' recognizes T .

Let Q' be the disjoint union of Q and the singleton $\{q_f\}$ and $F' = \{q_f\}$. Furthermore, let

$$\begin{aligned} \delta' = \delta \quad &\cap \quad (\Sigma \times Q^* \times Q) \\ &\cup \quad \{(a, \epsilon, q) \mid \delta(\perp, \epsilon, q') \text{ and } \delta(a, q', q)\} \\ &\cup \quad \{(a, w, q_f) \mid \delta(a, w, q) \text{ and } \delta(\top, q, q') \text{ and } q' \in F\}. \end{aligned}$$

□

9 Previous work

Work on tree automata and tree-regular languages can be divided into two categories, one dealing with ranked and the other with non-ranked alphabets.

The bulk of the literature belongs to the first category. Gécseg and Steinby [GS84] have written a comprehensive book on tree automata and tree transducers over ranked alphabets. In this standard reference trees are algebraic expressions and tree automata are algebras. The Handbook of Formal Languages [RS97] provides an updated and more concise version by the same authors.

Tree-regular languages over non-ranked alphabets seem to have been discovered first by Thatcher [Tha67a]. Two of Thatcher's papers [Tha67a, TW65]¹ state a number of results on tree automata that carry over directly from the theory of string automata. Neither of the two papers provides the proofs, though, which seem to be included in a technical report [Tha67b]. So far, we have not been able to track that report down.

The characterization of tree-regular languages as homomorphic images of tree-local sets essentially goes back to Thatcher [Tha67a] and was re-discovered by [].

Thatcher [Tha67a] indicates that results on semigroup automata, congruences, minimality, and decomposition also carry through from string languages to tree languages. This is, in 1969, cited as ongoing work; we have not been able to find any publications of this work, though.

¹Need to check [TW65].

Thatcher [Tha70] provides a light introduction into the theory of tree automata and tree-regular languages. This work indicates how the theory is applicable to context-free language problems, for example to structural equivalence. It also addresses tree transformations. The paper starts out with tree languages over non-ranked alphabets, but quickly restricts itself to the ranked-alphabet case.

Murata [Mur95] surveys definitions and results on tree-regular languages. His terminology goes back to Thatcher [Tha67a]. Murata extends his survey to forest languages since he is interested in tree transformations and forests often are the natural results of them. Murata covers tree automata, tree expressions, and regular tree grammars.

Barrero [Bar91] argues that in application areas such as pattern recognition and grammatical inference it is sensible to consider unranked alphabets. He develops a theory of tree languages over unranked alphabets. Although familiar with Thatcher's work, Barrero demands that for tree automata the number of transitions be finite. In this framework, tree languages are not closed under complement. Barrero also addresses tree grammars.

Moriya [Mor94] demonstrates that two-way tree automata are equipotent to one-way tree automata. The alphabet of tree labels is not ranked; however, like in Barrero's work [Bar91], the transition table is finite. Moriya's proof is based on crossing sequences, as is the corresponding proof for string automata. Moriya also addresses one-way and two-way pushdown tree automata; for this type of automata, two-way ones are more powerful.

Takahashi [Tak75] establishes a uniform mathematical framework to study regular sets of strings, trees, and forests, covering both ranked and unranked alphabets. She characterizes string-, tree-, and forest-regular sets as homomorphic images of local sets and as congruence-characterizable sets. Takahashi applies the theory of forest-regular languages to the structural study of context-free languages.

References

- [Bar91] A. Barrero. Unranked tree languages. *Pattern Recognition*, 24(1):9–18, 1991.
- [GS84] F. Gécseg and M. Steinby. *Tree Automata*. Akadémiai Kiadó, Budapest, 1984.
- [Mor94] E. Moriya. On two-way tree automata. *Information Processing Letters*, 50:117–121, 1994.
- [Mur95] M. Murata. Forest-regular languages and tree-regular languages. Working paper, 1995.
- [RS97] G. Rozenberg and A. Salomaa, editors. *Handbook of Formal Languages, Volume 3, Beyond Words*. Springer-Verlag, Berlin, Heidelberg, New York, 1997.

- [Tak75] M. Takahashi. Generalization of regular sets and their application to a study of context-free languages. *Information and Control*, 27(1):1–36, January 1975.
- [Tha67a] J.W. Thatcher. Characterizing derivation trees of context-free grammars through a generalization of finite automata theory. *Journal of Computer and System Sciences*, 1:317–322, 1967.
- [Tha67b] J.W. Thatcher. A further generalization of finite automata. Technical Report RC 1846, IBM Thomas J. Watson Research Center, Yorktown Heights, New York, 1967.
- [Tha70] J.W. Thatcher. There’s a lot more to finite automata theory than you would have thought. Technical Report RC 2852 (#13407), IBM Thomas J. Watson Research Center, Yorktown Heights, New York, 1970.
- [TW65] J.W. Thatcher and J.B. Wright. Abstract 65T-469. *Notices of the American Mathematical Society*, 12:820, 1965.